

Bloomberg

The Unix Philosophy for AI Agents

Filesystems as the Context Primitive

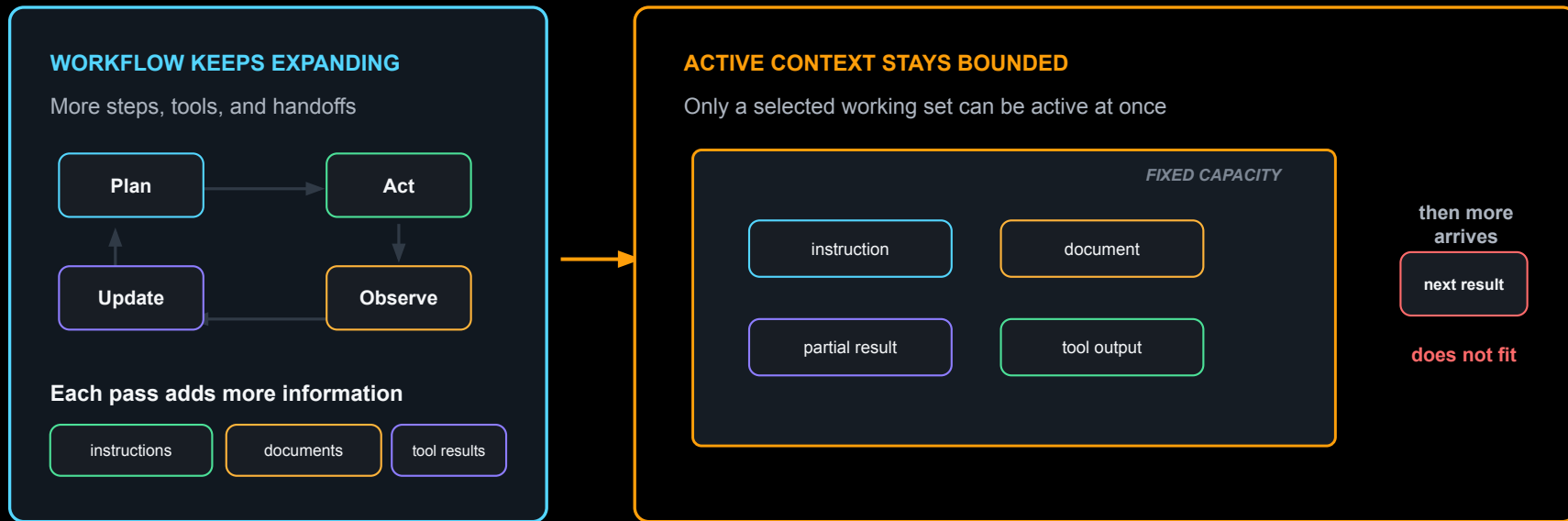
September 17, 2026

Daniel Temesgen, Senior Software Engineer, AI Foundational Platforms @ Bloomberg

Cannis Chan, Technical Product Manager, AI Platforms, Office of the CTO @ Bloomberg

The work is growing. The context window is not.

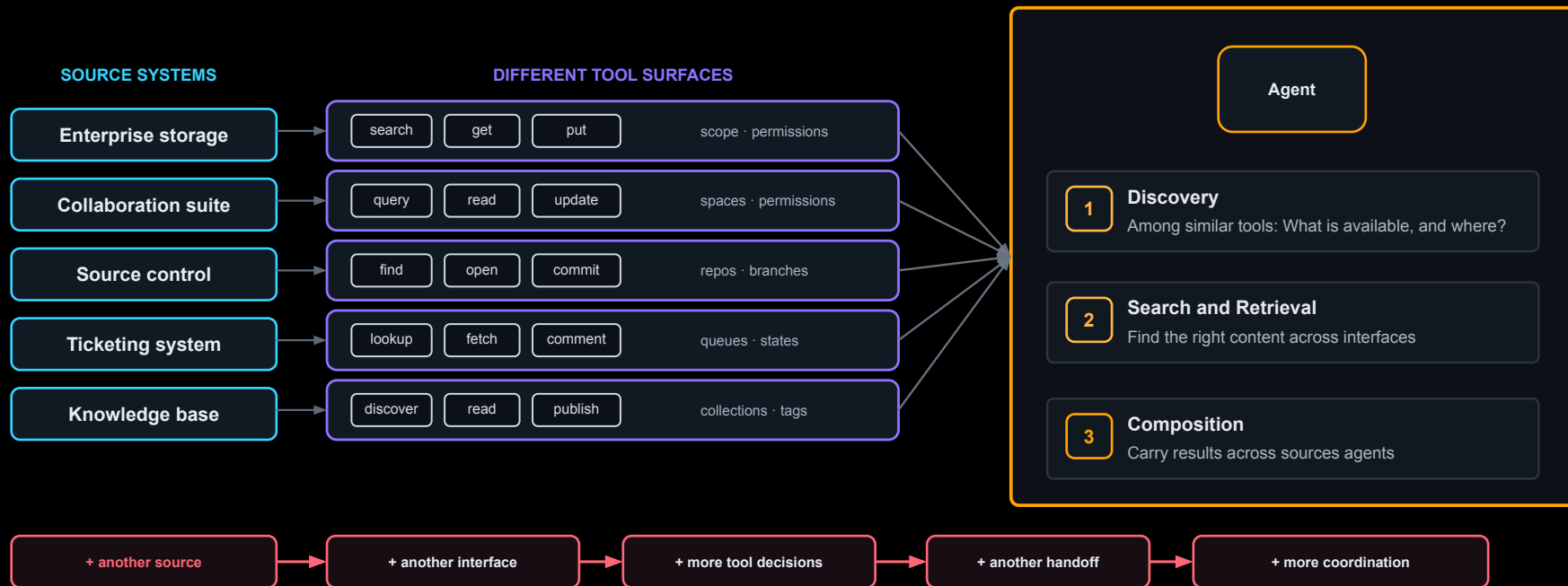
Longer workflows generate more information than should remain active at once.



Useful information must leave active context **without becoming lost.**

Every source adds decisions the workflow must get right

Agents must choose the correct tool, supply source-specific parameters, and translate each result for the next step.



More sources → more tool-selection errors, brittle handoffs, and custom integration work

Shared state needs boundaries

Multi-agent workflows need to share state, but not all state should be shared in the same way.



STATE TYPE	WHO CAN ACCESS?	HOW LONG?	WHO CAN WRITE?
Private working state plan, scratchpad	one agent	task	the creator
Shared coordination state intermediate output, artifact	workflow agents	conversation	coordinated writers

TOO ISOLATED

Agents cannot coordinate, handoffs lose context, work is repeated.

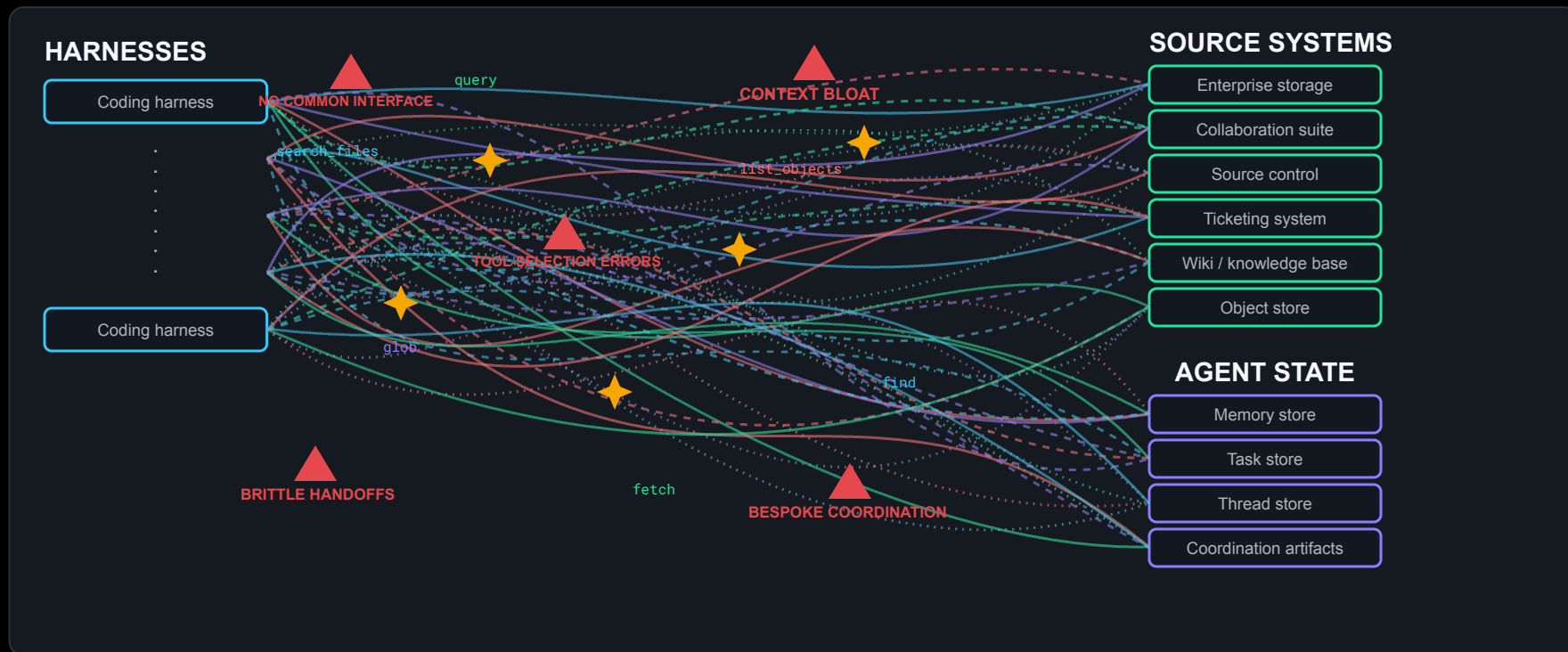
TOO EXPOSED

Private state leaks, and the wrong agents can read or change shared state.

Different state needs **different access, lifetime, and write rules**

The $M \times N$ Context Crisis

MCP standardised the transport and the shape of a tool call. Each source still names its own operations, so the agent resolves the differences at runtime.



This is an $M \times N$ problem. It needs one contract to become $M + N$. What should that contract carry?

Agents work one layer above the filesystem

TOO LOW — FUSE AND POSIX

Byte-level access

open, seek, read(offset, length). Every call is a round trip.

Efficient locally, costly remotely

Fine against a local disk. Expensive across a network, and the agent gains nothing from the extra precision.

Is this FUSE?

FUSE specifies byte-level access to a local mount.

THE LAYER WE WANT

The harnesses already work here

Their tools are not filesystem calls, and none of them operate on byte offsets.

Operations execute at the source

Today a source is mounted through to the client and treated as an ordinary filesystem, which makes transfer, search and write-back inefficient.

Is this File Uploads?

File Uploads declares user-selected file inputs. Its charter puts server-side content with Resources.

TOO HIGH — OPERATIONS MODELLED PER SOURCE

Large surface area

Standardising operations at the level each source already thinks in means every implementer builds a lot and keeps little flexibility.

Where this sits today

MCP resources carry read and list. Anything richer arrives as per-server tools, so the rich operations are never shared.

Is this HTTP again?

Where HTTP already carries the semantics, adopt them rather than specifying parallel ones.

Which layer should the protocol specify? That is what the working group exists to answer.

What we borrow from Unix

Unix is broad. We borrow one pattern, because models are already trained on these operations and the harnesses converged on them independently.

\$ CLASSIC UNIX PATTERN



small tools connected through a simple convention

- **Do one thing well**

`grep` searches. `sort` orders. Neither tool knows the other.

- **Discovery precedes reading**

`ls` names what is there. `grep` looks inside it.

- **Outputs become inputs**

`grep ERROR app.log | sort | uniq -c`

- **Text streams form a common interface**

A program reads a byte stream, without the producer's internals.

ADAPT

/ OUR AGENT ADAPTATION



steps compose through stable, named handoffs

- **Small operations**

`list(path), read(path), write(path, data), search(path, query), discover(path).`

- **Discovery and search run at the source**

Matching happens where the content already is, so only the results travel.

- **Operations compose over a path**

`discover /thread`, then `search /thread`, then `read` the matches. One vocabulary against any mount.

- **Stable, path-addressed handoffs**

`retrieve` writes `/thread/source.md`. `analyse` reads that path. Either step can be replaced, and models, tools and people read the same file.

Borrow the goal: **composability**. Adapt the mechanism: **paths and file operations**.

Every harness already ships a filesystem

CLIENT SIDE: HARNESSES

File operations exposed to the model

Claude Code

Read, Write, Edit, Glob,
Grep, Bash

Gemini CLI

read_file, write_file,
replace, glob,
search_file_content

Codex CLI

shell, apply_patch

THE MODEL

- **Computer use** is a headline benchmark. OSWorld-Verified leaders sit at ~85% (Sep 2026), vs ~72.7% (Feb 2026).
- Modern coding agents retrieve **just in time** with glob and grep instead of pre-computing an index.

"Smarter models require less prescriptive engineering, allowing agents to operate with more autonomy."

- Every custom tool surface we build has to be tuned again at the next model upgrade.

SERVER SIDE: SOURCES

- Enterprise Storage and Ticketing solutions each expose read, list, and search independently. Composition across them is manual.
- Tool definitions load up front: Anthropic reports **150,000 tokens** vs **2,000** when browsing a filesystem tree.
- **MCP skills (SEP-2640)** model skills as directories mounted into a virtual filesystem.

Resources are filesystem-shaped. Skills already had to extend them.

Working Group Accepted!

WHAT RESOURCES ALREADY GIVE US

- URI-addressed identity
- Read for a single resource
- Resource templates for unbounded trees
- Subscriptions and update notifications
- Partial metadata in listings

WHERE THE ABSTRACTION RUNS OUT

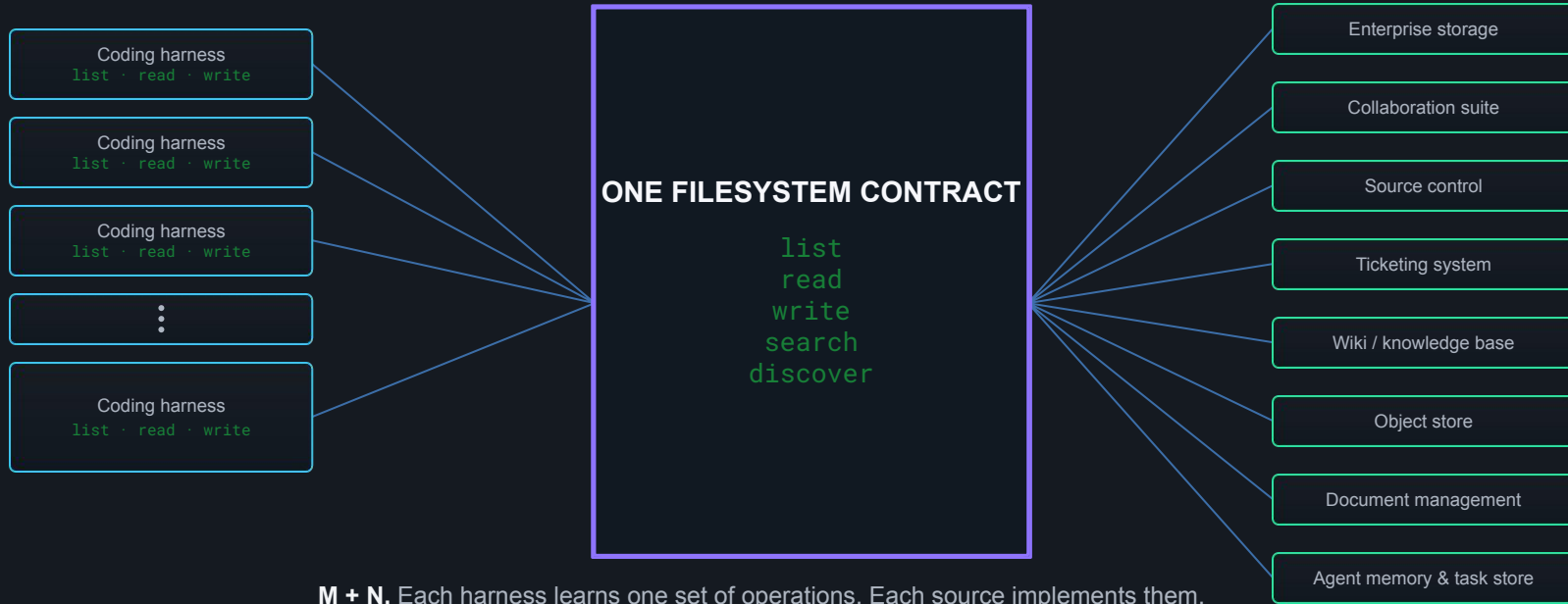
- Scoped listing under a prefix
- Search across resources
- Bulk reads in one call
- Metadata and revision information for a single path
- Create, update and delete on a path, with conditional writes
- Typed errors the agent can act on

Each feature that needs a filesystem extends resources again. **The abstraction should be defined once.**

Every harness learns one set of operations

One vocabulary, implemented by every source. The harness learns it once, and adding a source adds a directory.

WITH ONE FILESYSTEM CONTRACT



One navigation model for the agent. Each source keeps its own permissions, lifecycle and storage.

Real-World Alignment

Bloomberg and AI

Market
participants

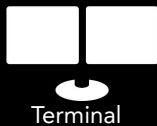


Buy-Side

Sell-Side

Corporations

Trusted
infrastructure for
high-stakes
financial decisions



Terminal



APIs & Analytics



Datasets

Powered by AI



Value to clients
and the global
financial market



Enhanced insights and transparency help power
decision-making by investment professionals

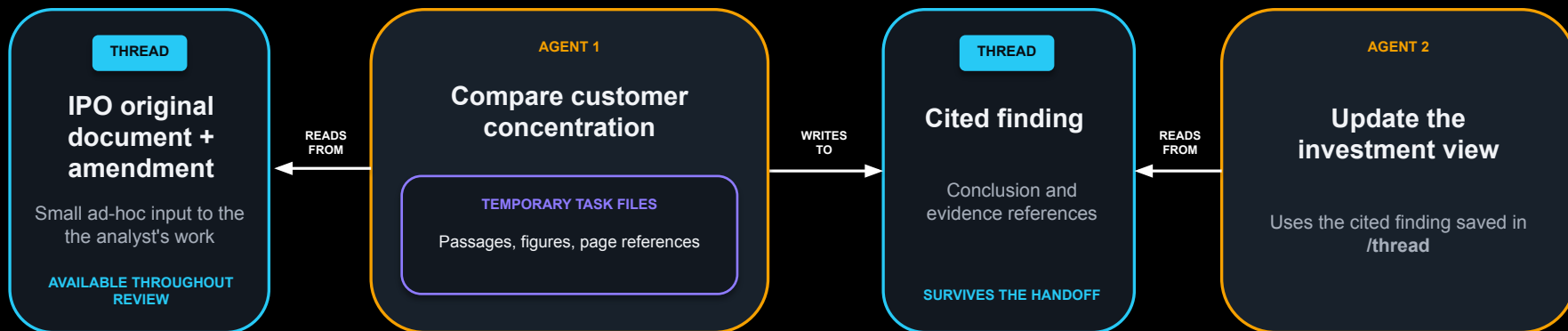


Agents can share information where it matters

A 'simple' review can create information with two different lifetimes.

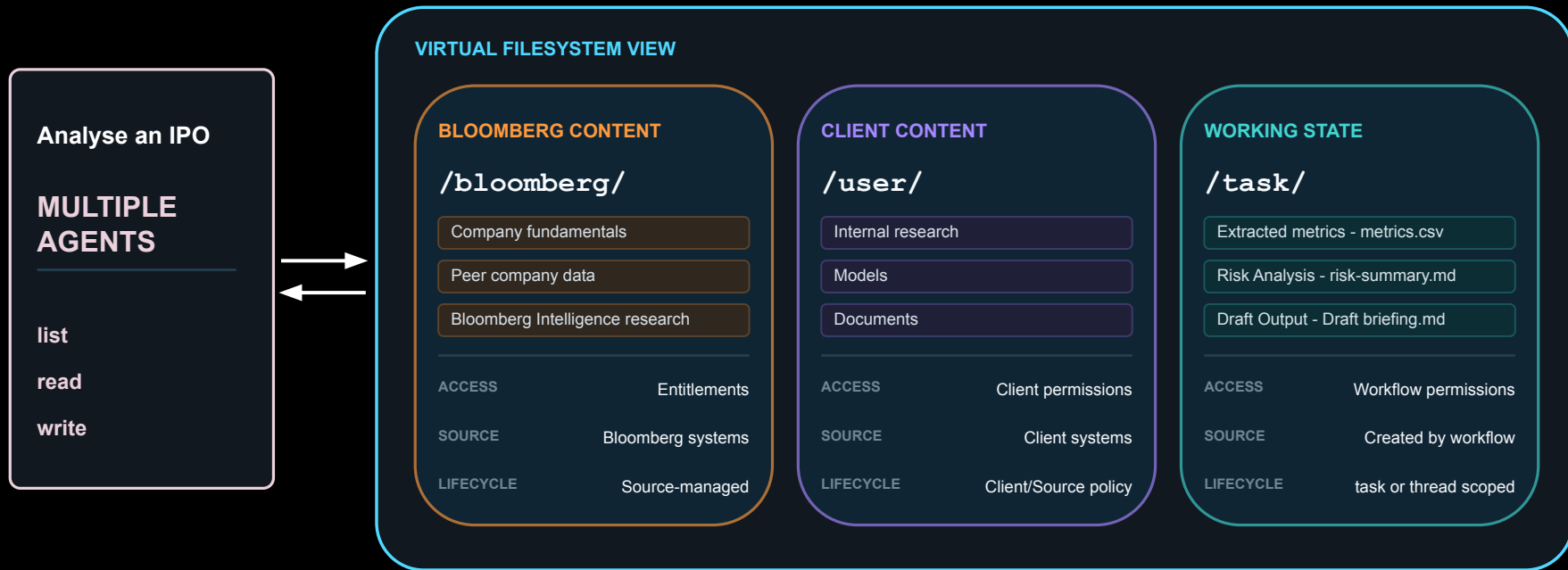
EQUITY ANALYST

“Did the amended filing change customer concentration risk?”



Bring your own content without flattening the rules

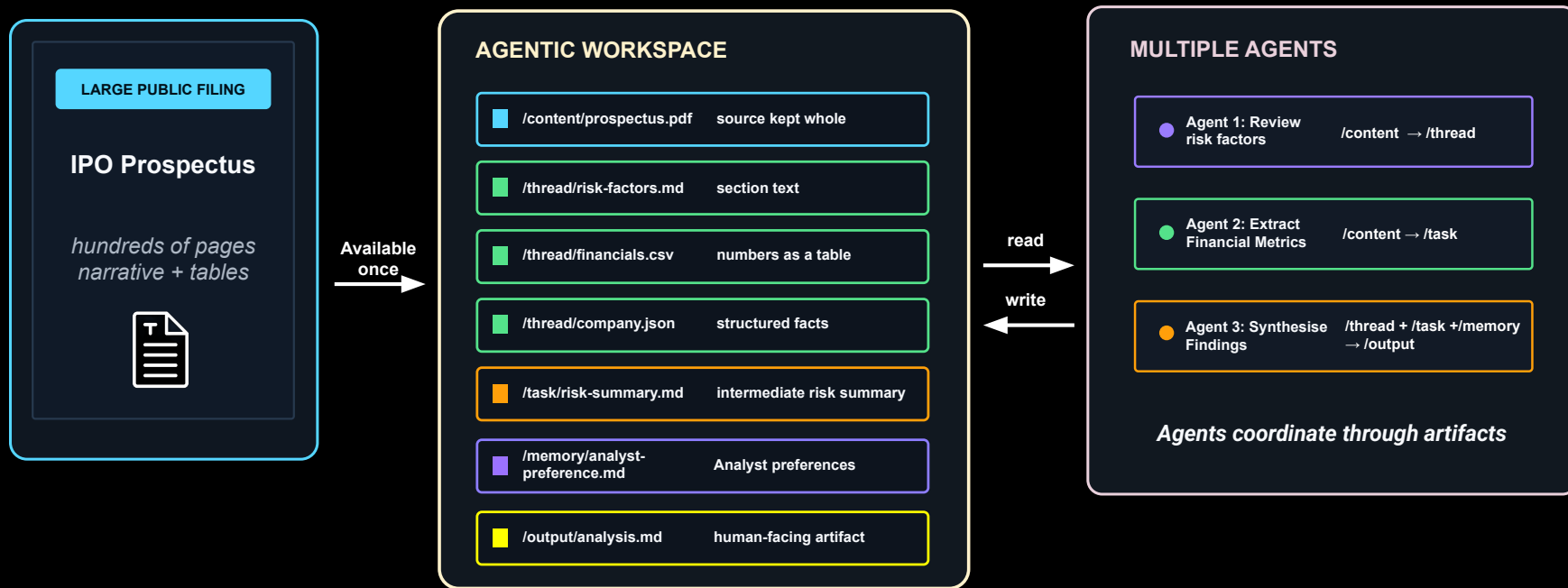
Research by an analyst may combine content and working state that follow very different rules.



One piece of work. One navigation model. Different contracts preserved.

A filesystem abstraction creates an agentic workspace

Agents compose the context they need, coordinate through named artifacts, and leave outputs others can use.



Join the Working Group!

A filesystem abstraction for MCP Resources. One contract for discovery, search, inspection and change.

IN SCOPE

DISCOVER AND SEARCH

Scoped listing under a prefix, with matching executed at the source so only results travel.

INSPECT

Metadata and revision information for a single path with `stat`

CHANGE SAFELY

Create, update and delete on a path, with optimistic concurrency and create-if-absent.

STAY CONSISTENT

How changes affect notifications, `lastModified`, TTLs and cached state.

OUT OF SCOPE

Not a parallel `files/*` API

Not local disk, mount or sandbox behaviour, and not POSIX

Not a new authorisation policy

Not resource submission, uploads or streaming. This work aligns with that group.

Get Involved

**Filesystems
WG Charter**



**Join Our
Discord**
filesystem-wg



**Check our Job
Board**



Thank you! Questions?

Daniel Temesgen

Senior Software Engineer

AI Foundational Platforms

Bloomberg



Cannis Chan

Technical Product Manager, AI Platforms

Office of the CTO

Bloomberg

